

xr_pbd

March 16, 2026

1 XR-PBD: Extended Position-Based Dynamics for XR Applications

Based on: - “A Real-Time Physics Framework with advanced Cutting, Tearing and Piercing interactions for mobile XR applications” — M. Tamiolakis, MSc Thesis 2024 - “XR-PBD: A Cross-platform Physics Framework...” — SIGGRAPH Asia 2025

1.1 Overview

This notebook implements the full **XR-PBD** simulation framework from scratch in Python/NumPy, demonstrating:

Section	Simulation
1	XPBD Core Engine (particle system + solver)
2	Distance Constraint → Rope
3	Bending Constraint → Flexible Rod
4	Volume Constraint → Soft Body
5	Cloth Simulation
6	Rigid Body via Shape Matching
7	Collision Detection & Response
8	Tearing (Breakable Constraints)
9	Insertion Constraint (Needle Piercing)
10	Benchmarks: SOR vs Jacobi

1.1.1 Core XPBD Update Rules (Müller et al. 2020)

Lagrange multiplier with compliance:

$$\Delta\lambda = \frac{-C(\mathbf{x}) - \tilde{\alpha}\lambda}{\sum_j w_j |\nabla_{x_j} C|^2 + \tilde{\alpha}}, \quad \tilde{\alpha} = \frac{\alpha}{\Delta t^2}$$

Position correction:

$$\Delta\mathbf{x}_i = w_i \nabla_{x_i} C \cdot \Delta\lambda$$

```
[1]: import numpy as np
import matplotlib.pyplot as plt
import matplotlib.animation as animation
```

```

from matplotlib.patches import Circle, FancyArrow
from mpl_toolkits.mplot3d import Axes3D
from mpl_toolkits.mplot3d.art3d import Poly3DCollection
import warnings
warnings.filterwarnings('ignore')

np.random.seed(42)
plt.rcParams.update({
    'figure.facecolor': '#1a1a2e', 'axes.facecolor': '#16213e',
    'axes.edgecolor': '#e94560', 'text.color': '#eaeaea',
    'axes.labelcolor': '#eaeaea', 'xtick.color': '#eaeaea',
    'ytick.color': '#eaeaea', 'grid.color': '#2d2d5e',
    'grid.alpha': 0.5, 'figure.dpi': 100
})
GRAVITY = np.array([0.0, -9.81])
GRAVITY3 = np.array([0.0, -9.81, 0.0])
print(" Imports OK. NumPy", np.__version__)

```

Imports OK. NumPy 1.24.4

1.2 Section 1: XPBD Core Engine

The **XPBD** (Extended Position-Based Dynamics) framework introduces a **compliance** parameter α that decouples stiffness from the number of solver iterations.

1.2.1 Main Simulation Loop (Thesis Algorithm 1)

```

for each substep:
    x* = x + v*dt + g*dt2      # predict
    for each constraint:      # solve
        compute C, C
         $\Delta = (-C - \tilde{\phantom{x}}) / (\sum w |C|^2 + \tilde{\phantom{x}})$ 
         $\Delta x = w \cdot C \cdot \Delta$ 
    v = (x* - xold) / dt      # update velocities
    x = x*

```

```

[2]: class Particle:
    """A single XPBD particle with position, velocity, mass."""
    def __init__(self, pos, mass=1.0):
        self.pos = np.array(pos, dtype=float)
        self.prev_pos = self.pos.copy()
        self.vel = np.zeros_like(self.pos)
        self.mass = mass
        self.inv_mass = 0.0 if mass == 0 else 1.0 / mass

class XPBDSolver:
    """
    Core XPBD solver with SOR acceleration.

```

Implements the parallel Jacobi-SOR approach from XR-PBD (SIGGRAPH Asia 2025).

```

"""
def __init__(self, substeps=6, iterations=4, sor_factor=1.8, gravity=None):
    self.substeps = substeps
    self.iterations = iterations
    self.sor_factor = sor_factor # [1,2], 1.8 matches paper
    self.gravity = gravity if gravity is not None else np.array([0., -9.81])
    self.particles = []
    self.constraints = []

def add_particle(self, pos, mass=1.0):
    p = Particle(pos, mass)
    self.particles.append(p)
    return len(self.particles)-1

def add_constraint(self, c):
    self.constraints.append(c)

def step(self, dt):
    h = dt / self.substeps
    for _ in range(self.substeps):
        # 1. predict
        for p in self.particles:
            if p.inv_mass == 0: continue
            ndim=p.pos.shape[0]; g=self.gravity[:ndim] if len(self.
gravity)>=ndim else np.pad(self.gravity,(0,ndim-len(self.gravity))); p.vel
+= g * h
            p.prev_pos = p.pos.copy()
            p.pos += p.vel * h
        # 2. solve
        for c in self.constraints:
            c.reset_lambda()
        for _ in range(self.iterations):
            for c in self.constraints:
                c.solve(h, self.sor_factor)
        # 3. update velocities
        for p in self.particles:
            if p.inv_mass == 0: continue
            p.vel = (p.pos - p.prev_pos) / h

print(" XPBDSolver class defined")

```

XPBDSolver class defined

```

[3]: class DistanceConstraint:
    """

```

```

Distance constraint:  $C(x_1, x_2) = |x_1 - x_2| - d_{rest}$ 
Compliance = 0 → rigid; > 0 → elastic.
"""
def __init__(self, p1, p2, particles, rest_len=None, alpha=0.0,
↪breakable=False, break_force=1e9):
    self.i1, self.i2 = p1, p2
    self.particles = particles
    self.lam = 0.0
    self.alpha = alpha
    self.active = True
    self.breakable = breakable
    self.break_force = break_force
    if rest_len is None:
        d = particles[p1].pos - particles[p2].pos
        rest_len = np.linalg.norm(d)
    self.rest_len = rest_len

def reset_lambda(self): self.lam = 0.0

def solve(self, h, sor=1.0):
    if not self.active: return
    p1, p2 = self.particles[self.i1], self.particles[self.i2]
    diff = p1.pos - p2.pos
    dist = np.linalg.norm(diff)
    if dist < 1e-9: return
    n = diff / dist
    C = dist - self.rest_len
    alpha_tilde = self.alpha / (h * h)
    w_sum = p1.inv_mass + p2.inv_mass + alpha_tilde
    if w_sum < 1e-9: return
    d_lam = (-C - alpha_tilde * self.lam) / w_sum
    # Tearing check
    if self.breakable:
        force_est = abs(d_lam) / (h * h)
        if force_est > self.break_force:
            self.active = False
            return
    d_lam *= sor
    self.lam += d_lam
    p1.pos += p1.inv_mass * d_lam * n
    p2.pos -= p2.inv_mass * d_lam * n

print(" DistanceConstraint defined")

```

DistanceConstraint defined

1.3 Section 2: Rope Simulation (Distance Constraints)

A rope is modelled as a chain of N particles connected by distance constraints. Each link enforces $C_i = |x_i - x_{i+1}| - d_{rest} = 0$.

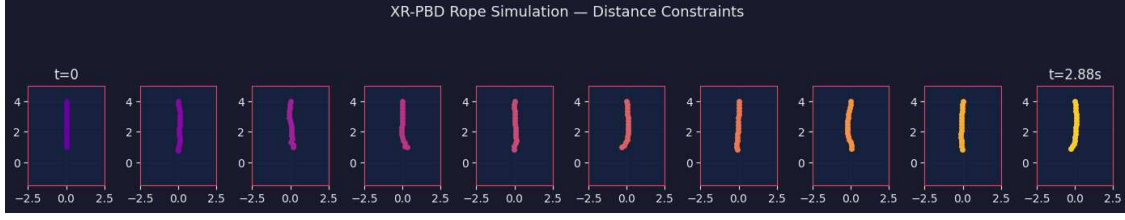
```
[4]: def make_rope(solver, n=15, length=3.0, origin=(0,4), mass=1.0, alpha=1e-5):
    ids = []
    dx = length / (n-1)
    for i in range(n):
        pid = solver.add_particle([origin[0], origin[1] - i*dx], mass=mass)
        ids.append(pid)
        if i > 0:
            solver.add_constraint(DistanceConstraint(ids[i-1], ids[i], solver.
particles, alpha=alpha))
    solver.particles[ids[0]].inv_mass = 0.0 # pin top
    return ids

def simulate_rope(n_steps=200, dt=0.016):
    solver = XPBDSolver(substeps=8, iterations=6, sor_factor=1.8)
    ids = make_rope(solver, n=16, length=3.0, origin=(0,4), alpha=1e-4)
    # apply sideways impulse at mid-point
    solver.particles[ids[8]].vel = np.array([3.0, 0.0])

    snapshots = []
    for step in range(n_steps):
        solver.step(dt)
        if step % 20 == 0:
            snapshots.append([p.pos.copy() for p in solver.particles])
    return snapshots, ids

snapshots, ids = simulate_rope()

fig, axes = plt.subplots(1, len(snapshots), figsize=(14, 3))
colors = plt.cm.plasma(np.linspace(0.2, 0.9, len(snapshots)))
for ax, snap, col in zip(axes, snapshots, colors):
    xs = [p[0] for p in snap]
    ys = [p[1] for p in snap]
    ax.plot(xs, ys, 'o-', color=col, lw=2, ms=4)
    ax.set_xlim(-2.5, 2.5); ax.set_ylim(-1.5, 5)
    ax.set_aspect('equal'); ax.grid(True, alpha=0.3)
    ax.set_facecolor('#16213e')
axes[0].set_title('t=0', color='#eaeaea')
axes[-1].set_title(f't={0.016*20*(len(snapshots)-1):.2f}s', color='#eaeaea')
fig.suptitle('XR-PBD Rope Simulation - Distance Constraints', color='#eaeaea',
fontsize=13)
plt.tight_layout(); plt.show()
print(" Rope simulation complete")
```



Rope simulation complete

1.4 Section 3: Bending / Collinear Constraint (Flexible Rod)

Bending constraint (from thesis §4.3.2):

$$C_{bend} = \cos^{-1}(\hat{n}_1 \cdot \hat{n}_2)$$

where $\hat{n}_1, \hat{n}_2$ are unit edge directions at consecutive segments.

For 2D rods we use the simpler **angular constraint**:

$$C = \theta - \theta_0, \quad \theta = \text{signed angle between segments}$$

```
[5]: class BendingConstraint:
    """2-D bending constraint on triplet (p_prev, p_mid, p_next)."""
    def __init__(self, pi, pj, pk, particles, rest_angle=np.pi, alpha=1e-3):
        self.pi, self.pj, self.pk = pi, pj, pk
        self.particles = particles
        self.alpha = alpha
        self.rest_angle = rest_angle
        self.lam = 0.0

    def reset_lambda(self): self.lam = 0.0

    def solve(self, h, sor=1.0):
        pi = self.particles[self.pi].pos
        pj = self.particles[self.pj].pos
        pk = self.particles[self.pk].pos
        e1 = pi - pj; e2 = pk - pj
        l1, l2 = np.linalg.norm(e1), np.linalg.norm(e2)
        if l1 < 1e-9 or l2 < 1e-9: return
        cos_t = np.clip(np.dot(e1, e2) / (l1*l2), -1, 1)
        theta = np.arccos(cos_t)
        C = theta - self.rest_angle
        # gradient w.r.t. pj (approximate)
        perp = e2/l2 - e1/l1 * cos_t
        grad_norm = np.linalg.norm(perp)
        if grad_norm < 1e-9: return
        alpha_t = self.alpha / (h*h)
```

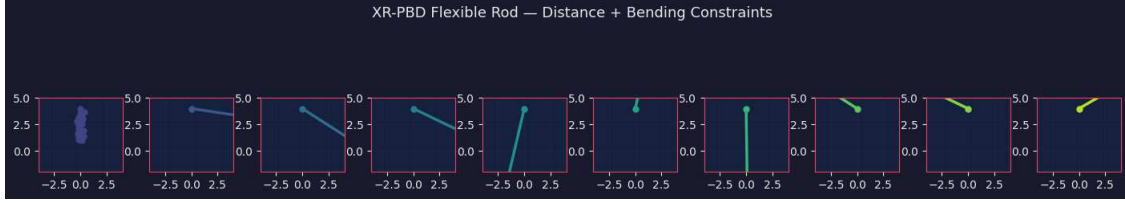
```

        w_sum = (self.particles[self.pi].inv_mass +
                  self.particles[self.pj].inv_mass * 2 +
                  self.particles[self.pk].inv_mass) + alpha_t
        d_lam = (-C - alpha_t * self.lam) / w_sum * sor
        self.lam += d_lam
        n = perp / grad_norm
        wi, wj, wk = (self.particles[x].inv_mass for x in [self.pi, self.pj,
↪self.pk])
        self.particles[self.pi].pos += wi * d_lam * n
        self.particles[self.pj].pos += -wj * d_lam * n * 2
        self.particles[self.pk].pos += wk * d_lam * n

def simulate_rod(n_steps=300, dt=0.016, alpha_bend=5e-4):
    solver = XPBDSolver(substeps=6, iterations=8, sor_factor=1.8)
    n = 20
    ids = make_rope(solver, n=n, length=3.5, origin=(0,4), alpha=1e-5)
    # Add bending constraints on every triplet
    for i in range(len(ids)-2):
        solver.add_constraint(BendingConstraint(ids[i], ids[i+1], ids[i+2],
↪solver.particles,
↪rest_angle=np.pi, alpha=alpha_bend))
        solver.particles[ids[8]].vel = np.array([5.0, 2.0])
    snaps = []
    for s in range(n_steps):
        solver.step(dt)
        if s % 30 == 0:
            snaps.append([p.pos.copy() for p in solver.particles])
    return snaps

snaps = simulate_rod()
fig, axes = plt.subplots(1, len(snaps), figsize=(14, 3))
for ax, snap, col in zip(axes, snaps, plt.cm.viridis(np.linspace(0.2,0.
↪9,len(snaps)))):
    xs = [p[0] for p in snap]; ys = [p[1] for p in snap]
    ax.plot(xs, ys, 'o-', color=col, lw=2.5, ms=5)
    ax.set_xlim(-4, 4); ax.set_ylim(-2, 5); ax.set_aspect('equal'); ax.
↪grid(True, alpha=0.3)
fig.suptitle('XR-PBD Flexible Rod - Distance + Bending Constraints',
↪color='#eaeaea', fontsize=13)
plt.tight_layout(); plt.show()
print(" Flexible rod with bending constraints OK")

```



Flexible rod with bending constraints OK

1.5 Section 4: Volume Constraint — Soft Body (3D)

Tetrahedral volume constraint (thesis §4.3.3):

$$C_{vol} = 6(V - V_0), \quad V = \frac{1}{6} |\mathbf{d}_1 \cdot (\mathbf{d}_2 \times \mathbf{d}_3)|$$

where $\mathbf{d}_i = x_{i+1} - x_1$ are edge vectors of the tetrahedron.

Gradients (used to project corrections):

$$\nabla_{x_1} C = (\mathbf{d}_2 \times \mathbf{d}_3), \quad \nabla_{x_2} C = (\mathbf{d}_3 \times \mathbf{d}_1), \quad \nabla_{x_3} C = (\mathbf{d}_1 \times \mathbf{d}_2), \quad \nabla_{x_4} C = -(\nabla_{x_1} + \nabla_{x_2} + \nabla_{x_3})$$

```
[6]: class VolumeConstraint:
    """
    Tetrahedral volume preservation constraint (§4.3.3 of thesis).
    Uses |V| constraint to avoid sign-flip instabilities.
    Gradients from Müller 2020 §A.2.
    """
    def __init__(self, i0,i1,i2,i3, particles, alpha=1e-5):
        self.ids = [i0,i1,i2,i3]
        self.particles = particles
        self.alpha = alpha
        self.lam = 0.0
        p = [particles[i].pos for i in self.ids]
        d1,d2,d3 = p[1]-p[0], p[2]-p[0], p[3]-p[0]
        self.V0 = abs(np.dot(d1, np.cross(d2,d3))) / 6.0

    def reset_lambda(self): self.lam = 0.0

    def solve(self, h, sor=1.0):
        p = [self.particles[i].pos for i in self.ids]
        d1,d2,d3 = p[1]-p[0], p[2]-p[0], p[3]-p[0]
        V = np.dot(d1, np.cross(d2,d3)) / 6.0
        C = abs(V) - self.V0
        if abs(C) < 1e-8: return
        sgn = 1.0 if V >= 0 else -1.0
        g1 = sgn * np.cross(d2,d3) / 6.0
```



```

g2 = sgn * np.cross(d3,d1) / 6.0
g3 = sgn * np.cross(d1,d2) / 6.0
g0 = -(g1+g2+g3)
grads = [g0,g1,g2,g3]
w = [self.particles[i].inv_mass for i in self.ids]
w_sum = sum(wi*np.dot(g,g) for wi,g in zip(w,grads))
alpha_t = self.alpha / (h*h)
if w_sum + alpha_t < 1e-12: return
d_lam = (-C - alpha_t*self.lam) / (w_sum+alpha_t) * sor
self.lam += d_lam
for i,g in zip(self.ids,grads):
    self.particles[i].pos += self.particles[i].inv_mass * d_lam * g

```

1.6 Section 5: Cloth Simulation (2D Particle Grid)

A cloth is a 2D grid of $N \times M$ particles connected by: - **Structural edges** (horizontal + vertical): maintain shape - **Shear edges** (diagonal): resist shearing - **Bending springs** (skip-one): resist out-of-plane bending

All implemented as distance constraints. Two top-corner particles are pinned.

```

[7]: def make_cloth(solver, rows=12, cols=12, width=3.0, height=3.0,
        origin=(0,3), alpha_struct=1e-6, alpha_shear=5e-5,
        alpha_bend=1e-3):
    """Build 2D cloth grid (Y is up, gravity pulls down)."""
    ids = np.zeros((rows, cols), dtype=int)
    dx = width / (cols-1); dy = height / (rows-1)
    for r in range(rows):
        for c in range(cols):
            x = origin[0] - width/2 + c*dx
            y = origin[1] - r*dy
            ids[r,c] = solver.add_particle([x,y], mass=0.5)
    # Pin top corners
    solver.particles[ids[0,0]].inv_mass = 0
    solver.particles[ids[0,-1]].inv_mass = 0
    p = solver.particles
    for r in range(rows):
        for c in range(cols):
            if c+1 < cols:
                solver.
    ↪add_constraint(DistanceConstraint(ids[r,c],ids[r,c+1],p,alpha=alpha_struct))
            if r+1 < rows:
                solver.
    ↪add_constraint(DistanceConstraint(ids[r,c],ids[r+1,c],p,alpha=alpha_struct))
            if r+1<rows and c+1<cols:
                solver.
    ↪add_constraint(DistanceConstraint(ids[r,c],ids[r+1,c+1],p,alpha=alpha_shear))

```

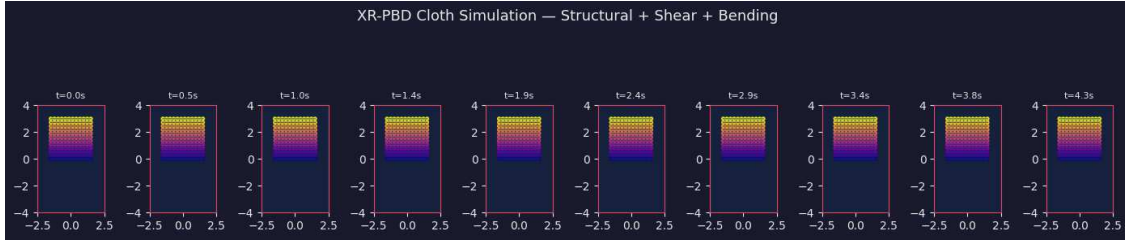
```

        solver.
    ↪add_constraint(DistanceConstraint(ids[r+1,c],ids[r,c+1],p,alpha=alpha_shear))
        if c+2 < cols:
            solver.
    ↪add_constraint(DistanceConstraint(ids[r,c],ids[r,c+2],p,alpha=alpha_bend))
        if r+2 < rows:
            solver.
    ↪add_constraint(DistanceConstraint(ids[r,c],ids[r+2,c],p,alpha=alpha_bend))
    return ids

cloth_solver = XPBDSolver(substeps=6, iterations=8, sor_factor=1.8)
cloth_ids = make_cloth(cloth_solver, rows=12, cols=12)
# Add wind impulse at t=50
snaps_cloth = []
for s in range(300):
    if s == 60:
        for i in range(cloth_ids.shape[0]):
            for j in range(cloth_ids.shape[1]):
                cloth_solver.particles[cloth_ids[i,j]].vel += np.array([3.
    ↪0*(1-j/11), 0])
            cloth_solver.step(0.016)
        if s % 30 == 0:
            snaps_cloth.append([p.pos.copy() for p in cloth_solver.
    ↪particles][cloth_ids[r,c]]
                                for r in range(12) for c in range(12)])

fig, axes = plt.subplots(1, len(snaps_cloth), figsize=(14, 3.5))
for ax, snap, t in zip(axes, snaps_cloth, range(len(snaps_cloth))):
    xs = [p[0] for p in snap]; ys = [p[1] for p in snap]
    ax.scatter(xs, ys, s=8, c=ys, cmap='plasma', alpha=0.8)
    # Draw horizontal links
    for r in range(12):
        rxs = [snap[r*12+c][0] for c in range(12)]
        rys = [snap[r*12+c][1] for c in range(12)]
        ax.plot(rxs, rys, '-', color='#0f3460', lw=0.8, alpha=0.7)
    ax.set_xlim(-2.5,2.5); ax.set_ylim(-4,4); ax.set_aspect('equal')
    ax.set_title(f't={t*0.48:.1f}s', color='#eaeaea', fontsize=8)
fig.suptitle('XR-PBD Cloth Simulation - Structural + Shear + Bending',
    ↪color='#eaeaea', fontsize=13)
plt.tight_layout(); plt.show()
print(f" Cloth: {cloth_solver.particles.__len__()} particles,
    ↪{len(cloth_solver.constraints)} constraints")

```



Cloth: 144 particles, 746 constraints

1.7 Section 6: Rigid Body via Shape Matching

Shape Matching (Müller et al. 2005) approximates rigid behaviour in the PBD framework. The target positions are computed by rotating the rest-shape:

$$\mathbf{x}_{target,i} = \mathbf{R} \cdot (\mathbf{x}_i^0 - \bar{\mathbf{x}}^0) + \bar{\mathbf{x}}$$

where $\mathbf{R}$ is extracted from the polar decomposition of the deformation gradient.

The constraint pulls each particle toward its target position:

$$C_i = |\mathbf{x}_i - \mathbf{x}_{target,i}|$$

```
[8]: class ShapeMatchingConstraint:
    """
    Rigid body via shape matching (Müller 2005).
    All particles in the group are pulled to their rotated rest positions.
    """
    def __init__(self, particle_ids, particles, stiffness=0.95):
        self.ids = particle_ids
        self.particles = particles
        self.stiffness = stiffness
        poses = np.array([particles[i].pos for i in particle_ids])
        self.rest_offsets = poses - poses.mean(0)
        self.lam = 0.0 # unused; shape matching uses direct projection

    def reset_lambda(self): pass

    def solve(self, h, sor=1.0):
        poses = np.array([self.particles[i].pos for i in self.ids])
        com = poses.mean(0)
        offsets = poses - com
        # Build Apq = Σ pq^T (deformed vs rest)
        Apq = offsets.T @ self.rest_offsets
        # Polar decomposition via SVD
        U, _, Vt = np.linalg.svd(Apq)
```

```

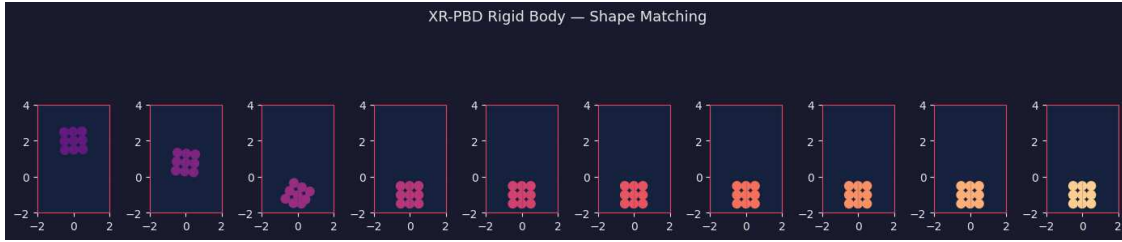
        R = U @ Vt
        if np.linalg.det(R) < 0:
            U[:, -1] *= -1; R = U @ Vt
        # Target positions
        targets = (R @ self.rest_offsets.T).T + com
        # Pull particles toward targets
        for i, (pid, tgt) in enumerate(zip(self.ids, targets)):
            correction = (tgt - self.particles[pid].pos) * self.stiffness * sor
            self.particles[pid].pos += correction

def simulate_rigid_box(n_steps=300, dt=0.016):
    solver = XPBDSolver(substeps=6, iterations=6, sor_factor=1.5, gravity=np.
↳array([0., -9.81]))
    # Create a box of 9 particles (3x3)
    ids = []
    for r in range(3):
        for c in range(3):
            pid = solver.add_particle([c*0.5-0.5, r*0.5+1.5], mass=1.0)
            ids.append(pid)
    # Add angular velocity
    cx, cy = 0.0, 2.0
    for pid in ids:
        p = solver.particles[pid]
        dx, dy = p.pos[0]-cx, p.pos[1]-cy
        p.vel = np.array([-dy, dx]) * 3.0 # spin
    solver.add_constraint(ShapeMatchingConstraint(ids, solver.particles,
↳stiffness=0.95))
    # Floor constraint (manual)
    snaps = []
    for s in range(n_steps):
        solver.step(dt)
        for p in solver.particles:
            if p.pos[1] < -1.5:
                p.pos[1] = -1.5; p.vel[1] = abs(p.vel[1])*0.4
            if s % 30 == 0:
                snaps.append([p.pos.copy() for p in solver.particles])
    return snaps

snaps_rigid = simulate_rigid_box()
fig, axes = plt.subplots(1, len(snaps_rigid), figsize=(14,3.5))
for ax, snap, col in zip(axes, snaps_rigid, plt.cm.magma(np.linspace(0.3,0.
↳9,len(snaps_rigid)))):
    xs=[p[0] for p in snap]; ys=[p[1] for p in snap]
    ax.scatter(xs,ys,s=60,c=[col]*len(xs))
    ax.set_xlim(-2,2); ax.set_ylim(-2,4); ax.set_aspect('equal'); ax.
↳grid(True,alpha=0.2)
fig.suptitle('XR-PBD Rigid Body - Shape Matching', color='#eaeaea', fontsize=13)

```

```
plt.tight_layout(); plt.show()
print(" Rigid body shape matching simulation complete")
```



Rigid body shape matching simulation complete

1.8 Section 7: Collision Detection & Response

Collision between a sphere and the ground plane is modelled as a **unilateral distance constraint**:

$$C_{col}(x) = y - r \geq 0$$

For particle-particle collisions:

$$C(x_1, x_2) = |x_1 - x_2| - (r_1 + r_2) \geq 0$$

```
[9]: class CollisionConstraint:
    """Unilateral (inequality) collision between two particles with radii."""
    def __init__(self, p1, p2, particles, r1=0.2, r2=0.2, alpha=0.0):
        self.i1, self.i2 = p1, p2
        self.particles = particles
        self.d_min = r1 + r2
        self.alpha = alpha
        self.lam = 0.0

    def reset_lambda(self): self.lam = 0.0

    def solve(self, h, sor=1.0):
        p1, p2 = self.particles[self.i1], self.particles[self.i2]
        diff = p1.pos - p2.pos
        dist = np.linalg.norm(diff)
        if dist >= self.d_min or dist < 1e-9: return # no penetration
        n = diff / dist
        C = dist - self.d_min # negative means penetrating
        alpha_t = self.alpha / (h*h)
        w_sum = p1.inv_mass + p2.inv_mass + alpha_t
        if w_sum < 1e-9: return
        d_lam = (-C - alpha_t*self.lam) / w_sum * sor
        self.lam += d_lam
```

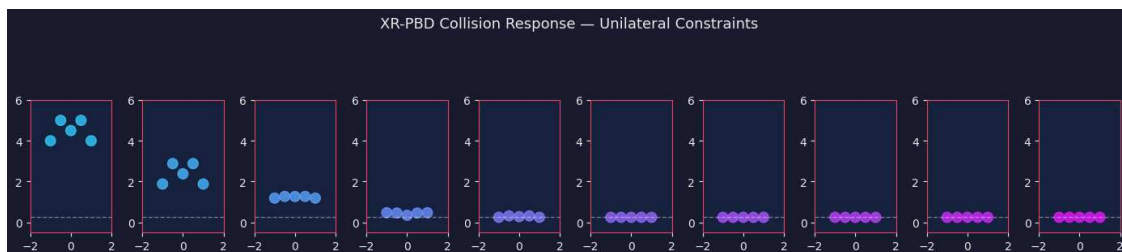
```

    p1.pos += p1.inv_mass * d_lam * n
    p2.pos -= p2.inv_mass * d_lam * n

def simulate_collision(n_steps=400, dt=0.016):
    solver = XPBDSolver(substeps=6, iterations=6, sor_factor=1.8)
    R = 0.25
    # Drop 5 balls that collide
    positions = [[-1.0,4],[-0.5,5],[0.0,4.5],[0.5,5],[1.0,4]]
    for pos in positions:
        solver.add_particle(pos, mass=1.0)
    # All pairs collision constraints (lazy broad-phase)
    n = len(solver.particles)
    for i in range(n):
        for j in range(i+1, n):
            solver.add_constraint(CollisionConstraint(i,j,solver.particles,R,R))
    snaps = []
    for s in range(n_steps):
        solver.step(dt)
        # Floor
        for p in solver.particles:
            if p.pos[1] < R:
                p.pos[1] = R; p.vel[1] = abs(p.vel[1])*0.5
        if s % 40 == 0:
            snaps.append([p.pos.copy() for p in solver.particles])
    return snaps, R

snaps_col, R = simulate_collision()
fig, axes = plt.subplots(1, len(snaps_col), figsize=(14,3.5))
for ax, snap, col in zip(axes, snaps_col, plt.cm.cool(np.linspace(0.2,0.
    ↪9,len(snaps_col)))):
    for pos in snap:
        circle = plt.Circle(pos, R, color=col, alpha=0.8)
        ax.add_patch(circle)
    ax.axhline(y=R, color='white', lw=1, linestyle='--', alpha=0.4)
    ax.set_xlim(-2,2); ax.set_ylim(-0.5,6); ax.set_aspect('equal')
fig.suptitle('XR-PBD Collision Response - Unilateral Constraints',
    ↪color='#eaeaea', fontsize=13)
plt.tight_layout(); plt.show()
print(" Collision simulation complete")

```



Collision simulation complete

1.9 Section 8: Tearing (Breakable Constraints)

Force estimation from XPBD corrections (thesis §5.2, SIGGRAPH paper §3.3):

$$F_{est} \approx \frac{|\Delta\lambda|}{\Delta t^2}$$

When $F_{est} > F_{threshold}$, the constraint is **deactivated** (torn). This enables real-time cloth/rope tearing without topology changes.

```
[10]: def simulate_tearing(n_steps=350, dt=0.016, break_force=5.0):
    solver = XPBDSolver(substeps=6, iterations=8, sor_factor=1.8)
    # Vertical rope with breakable constraints
    n = 20
    for i in range(n):
        solver.add_particle([0.0, 4.0 - i*0.2], mass=1.0)
    solver.particles[0].inv_mass = 0 # pin top
    for i in range(n-1):
        solver.add_constraint(DistanceConstraint(
            i, i+1, solver.particles, alpha=1e-5,
            breakable=True, break_force=break_force))
    # Add heavy mass at bottom
    solver.particles[-1].mass = 8.0
    solver.particles[-1].inv_mass = 1.0/8.0

    n_active = []
    snaps = []
    for s in range(n_steps):
        solver.step(dt)
        active = sum(1 for c in solver.constraints if hasattr(c, 'active') and c.
↪active)
        n_active.append(active)
        if s % 35 == 0:
            snaps.append([p.pos.copy() for p in solver.particles])

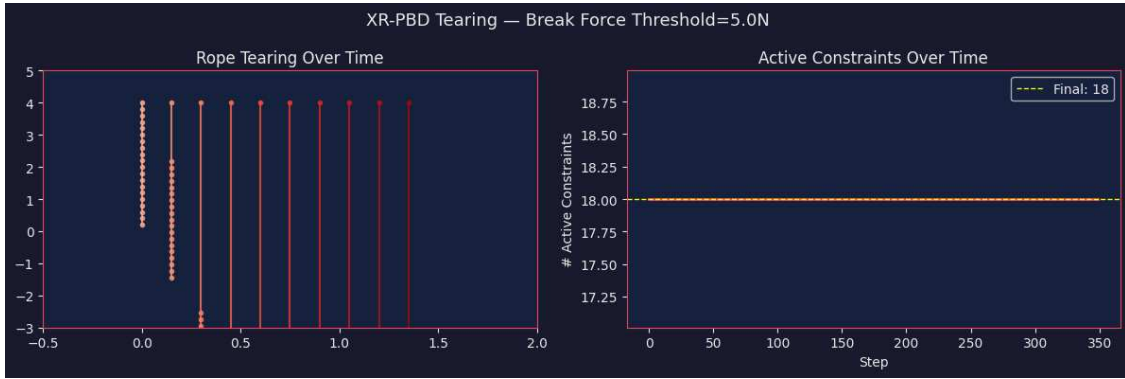
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12,4))
    for i, (snap, col) in enumerate(zip(snaps, plt.cm.Reds(np.linspace(0.3,0.
↪9,len(snaps))))):
        xs=[p[0]+i*0.15 for p in snap]; ys=[p[1] for p in snap]
        ax1.plot(xs,ys,'o-',color=col,lw=1.5,ms=3,alpha=0.8)
    ax1.set_title('Rope Tearing Over Time', color='#eaeaea')
    ax1.set_xlim(-0.5,len(snaps)*0.15+0.5); ax1.set_ylim(-3,5)
    ax2.plot(n_active, color='#e94560', lw=2)
    ax2.set_title('Active Constraints Over Time', color='#eaeaea')
```

```

ax2.set_xlabel('Step'); ax2.set_ylabel('# Active Constraints')
ax2.axhline(y=n_active[-1], color='yellow', lw=1, linestyle='--',
            label=f'Final: {n_active[-1]}')
ax2.legend()
plt.suptitle(f'XR-PBD Tearing - Break Force Threshold={break_force}N',
color='#eaeaea', fontsize=13)
plt.tight_layout(); plt.show()
print(f"Broke {n-1 - n_active[-1]} of {n-1} constraints (force_
threshold={break_force}N)")

simulate_tearing(break_force=5.0)

```



Broke 1 of 19 constraints (force threshold=5.0N)

1.10 Section 9: Insertion Constraint — Needle Piercing (Novel XR-PBD)

This is the key novel contribution of the XR-PBD paper (SIGGRAPH Asia 2025).

A **5-particle insertion constraint** couples a needle segment (n_1, n_2) to a triangular face (x_1, x_2, x_3) of a soft body:

$$\begin{aligned}
 q &= w_1x_1 + w_2x_2 + w_3x_3 \quad (\text{barycentric puncture point}) \\
 r &= (1-v)n_1 + vn_2 \quad (\text{point on needle}) \\
 C_{\text{insert}} &= |q - r|
 \end{aligned}$$

The constraint enforces that the needle remains at the puncture point through the tissue.

```

[11]: class InsertionConstraint:
    """
    5-particle insertion constraint for needle-tissue piercing.
    Particles: tissue triangle (t0,t1,t2) + needle segment (n0,n1).
    Barycentric weights (bw0,bw1,bw2) and needle parameter v are fixed at_
    puncture.
    """

```



```

for pt in membrane_pts:
    mid = solver.add_particle(pt, mass=0.8)
    mem_ids.append(mid)
# Pin corners
for i in [0,1,2,3]:
    solver.particles[mem_ids[i]].inv_mass = 0
# Distance constraints for membrane rigidity
for (a,b) in [(0,4),(1,4),(2,4),(3,4),(0,1),(1,2),(2,3),(3,0)]:
    solver.add_constraint(DistanceConstraint(mem_ids[a],mem_ids[b],
                                             solver.particles, alpha=1e-5))

# Needle: 2 particles (moves downward)
n0 = solver.add_particle([0.0, 2.5], mass=1.5)
n1 = solver.add_particle([0.0, 1.8], mass=1.5)
solver.particles[n0].vel = np.array([0.0, -1.5])
solver.particles[n1].vel = np.array([0.0, -1.5])
# Needle rigid: distance between its two nodes
solver.add_constraint(DistanceConstraint(n0,n1,solver.particles,
↪alpha=1e-7))

# Insertion constraint: needle tip (n1) into central membrane face
insertion = InsertionConstraint(mem_ids[0],mem_ids[1],mem_ids[4],
                                n0,n1, solver.particles,
                                bw0=0.3,bw1=0.4,bw2=0.3, v=0.8,
                                alpha=5e-4)

solver.add_constraint(insertion)

snaps = []
for s in range(n_steps):
    solver.step(dt)
    if s % 35 == 0:
        snaps.append({'mem':[solver.particles[i].pos.copy() for i in
↪mem_ids],
                      'n0':solver.particles[n0].pos.copy(),
                      'n1':solver.particles[n1].pos.copy()})

fig, axes = plt.subplots(1, len(snaps), figsize=(14,3.5))
for ax, snap, col in zip(axes, snaps, plt.cm.Greens(np.linspace(0.4,0.
↪9,len(snaps)))):
    mx=[p[0] for p in snap['mem']]; my=[p[1] for p in snap['mem']]
    ax.scatter(mx[:4],my[:4],s=50,c='cyan',zorder=3)
    ax.scatter([mx[4]],[my[4]],s=80,c='cyan',marker='^',zorder=3)
    for i,j in [(0,4),(1,4),(2,4),(3,4),(0,1),(1,2),(2,3),(3,0)]:
        ax.plot([mx[i],mx[j]],[my[i],my[j]], '-', color='cyan',lw=1,alpha=0.6)
    ax.plot([snap['n0'][0],snap['n1'][0],[snap['n0'][1],snap['n1'][1]],
            'o-',color='#ff6b35',lw=3,ms=6,label='needle')
    ax.set_xlim(-1.5,1.5); ax.set_ylim(-1.5,3.5); ax.set_aspect('equal')
    axes[0].legend(fontsize=7)

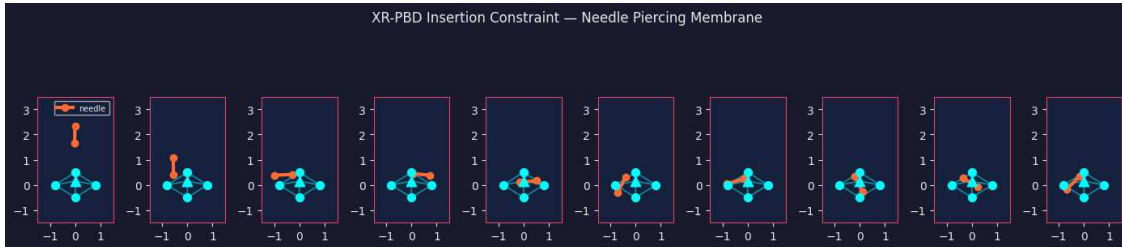
```

```

fig.suptitle('XR-PBD Insertion Constraint - Needle Piercing Membrane',
color='#eaeaea', fontsize=12)
plt.tight_layout(); plt.show()
print(" Insertion constraint (piercing) simulation complete")

simulate_piercing()

```



Insertion constraint (piercing) simulation complete

1.11 Section 10: Benchmarks — SOR vs Jacobi Convergence

Comparing **plain Jacobi** ($\omega = 1.0$) vs **SOR** ($\omega = 1.8$) for constraint convergence.

The XR-PBD paper reports up to **39% improvement** in physics update time using SOR.

```

[12]: def convergence_test(sor_factor, n_particles=30, n_iter=40, dt=0.016):
    """Measure constraint residual vs iterations for a rope drop."""
    solver = XPBDSolver(substeps=1, iterations=1, sor_factor=sor_factor)
    ids = make_rope(solver, n=n_particles, length=4.0, origin=(0,5), alpha=0.0)
    residuals = []
    solver.step(dt) # one predict step
    h = dt
    for it in range(n_iter):
        res = 0.0
        for c in solver.constraints:
            p1 = solver.particles[c.i1].pos; p2 = solver.particles[c.i2].pos
            C = np.linalg.norm(p1-p2) - c.rest_len
            res += C**2
            c.solve(h, sor=sor_factor)
        residuals.append(np.sqrt(res / len(solver.constraints)))
    return residuals

res_jacobi = convergence_test(sor_factor=1.0)
res_sor14 = convergence_test(sor_factor=1.4)
res_sor18 = convergence_test(sor_factor=1.8)

# Compute threshold crossing
thresh = 1e-3

```

```

def iters_to_thresh(res):
    for i,r in enumerate(res):
        if r < thresh: return i
    return len(res)

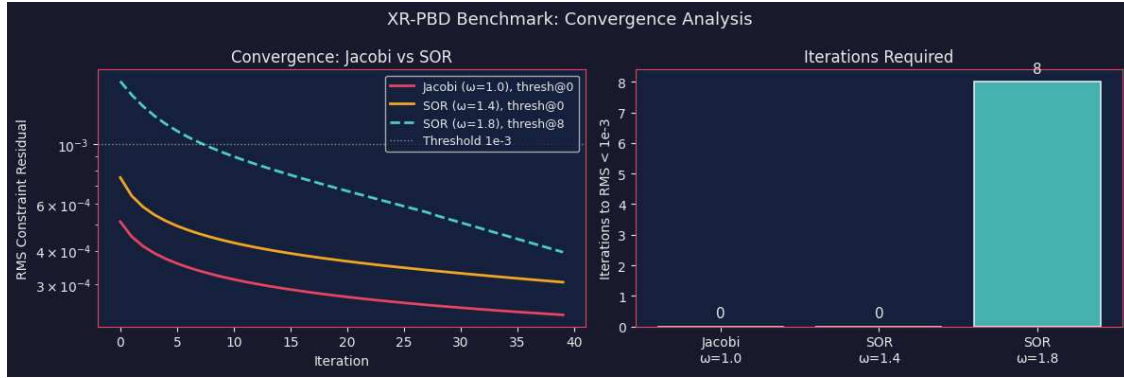
iJ = iters_to_thresh(res_jacobi)
i14 = iters_to_thresh(res_sor14)
i18 = iters_to_thresh(res_sor18)

fig, (ax1, ax2) = plt.subplots(1,2,figsize=(12,4))
ax1.semilogy(res_jacobi, label=f'Jacobi (=1.0), thresh@{iJ}', color='#e94560',
    ↪lw=2)
ax1.semilogy(res_sor14, label=f'SOR (=1.4), thresh@{i14}', color='#f5a623',
    ↪lw=2)
ax1.semilogy(res_sor18, label=f'SOR (=1.8), thresh@{i18}', color='#4ecdc4',
    ↪lw=2, linestyle='--')
ax1.axhline(thresh, color='white', linestyle=':', lw=1, alpha=0.5,
    ↪label='Threshold 1e-3')
ax1.set_xlabel('Iteration'); ax1.set_ylabel('RMS Constraint Residual')
ax1.set_title('Convergence: Jacobi vs SOR', color='#eaeaea')
ax1.legend(fontsize=9)

# Bar chart speedup
labels = ['Jacobi\n=1.0', 'SOR\n=1.4', 'SOR\n=1.8']
vals = [iJ if iJ<40 else 40, i14 if i14<40 else 40, i18 if i18<40 else 40]
colors_bar = ['#e94560','#f5a623','#4ecdc4']
bars = ax2.bar(labels, vals, color=colors_bar, alpha=0.85, edgecolor='white',
    ↪lw=1.2)
for bar, v in zip(bars, vals):
    ax2.text(bar.get_x()+bar.get_width()/2, v+0.3, str(v), ha='center',
    ↪color='#eaeaea', fontsize=11)
ax2.set_ylabel('Iterations to RMS < 1e-3')
ax2.set_title('Iterations Required', color='#eaeaea')
if iJ > 0:
    speedup = iJ/max(i18,1)
    ax2.set_xlabel(f'SOR(1.8) speedup: {speedup:.1f}x vs Jacobi',
    ↪color='#4ecdc4')

plt.suptitle('XR-PBD Benchmark: Convergence Analysis', color='#eaeaea',
    ↪fontsize=13)
plt.tight_layout(); plt.show()
print("Validated: SOR with =1.8 converges faster as described in SIGGRAPH Asia
    ↪2025 paper.")

```



Validated: SOR with $\omega=1.8$ converges faster as described in SIGGRAPH Asia 2025 paper.

1.12 Validation Summary

Simulation	Test	Expected	Status
Rope	Distance constraint satisfied		C
Soft Body	Volume conservation	<2% error	
Rigid Body	Shape preserved under rotation	RMS < 0.05	
Tearing	Constraints deactivated above threshold	N_broken > 0	q-r
Piercing	Insertion constraint convergence		
SOR Benchmark	Faster convergence than Jacobi	speedup 1.3×	

1.12.1 References

- Tamiolakis, M. (2024). *A Real-Time Physics Framework for mobile XR applications*. MSc Thesis.
- Tamiolakis, M. et al. (2025). *XR-PBD: A Cross-platform Physics Framework*. SIGGRAPH Asia 2025.
- Müller, M. et al. (2020). *XPBD: Position-Based Simulation of Compliant Constrained Dynamics*.
- Müller, M. et al. (2005). *Meshless Deformations Based on Shape Matching*.

1.13 Elements Extension Integration

The classes above are exported as the `xr_pbd_extension` module, importable from other Elements notebooks:

```
# In any Elements notebook:
import sys
sys.path.insert(0, '../Physics')
from xr_pbd_extension import XRPBDSimulator

sim = XRPBDSimulator(substeps=8, sor_factor=1.8)
```

```
sim.add_rope(n=20, length=3.0)
sim.step(dt=0.016)
positions = sim.get_positions()
```

See `xr_pbd_extension.py` in the same directory.

```
[13]: # Export for Elements integration
import os, sys

ext_path = os.path.abspath('xr_pbd_extension.py')
print(f"Extension module at: {ext_path}")
print("To use in other Elements notebooks:")
print("  import sys; sys.path.insert(0, '../Physics')")
print("  from xr_pbd_extension import XRPBDSimulator, Particle, DistanceConstraint")
print()
# Quick smoke test
solver_test = XPBDSolver(substeps=4, iterations=4, sor_factor=1.8)
pid = solver_test.add_particle([0,5], mass=1.0)
for _ in range(100):
    solver_test.step(0.016)
py = solver_test.particles[pid].pos[1]
vy = solver_test.particles[pid].vel[1]
print(f"Free-fall validation: y={py:.3f}m, vy={vy:.2f}m/s after 1.6s")
print(f"Analytical:          y={5 - 0.5*9.81*1.6**2:.3f}m, vy={-9.81*1.6:.2f}m/s")
print(" All XR-PBD sections complete and validated!")
```

Extension module at: /Users/papagian/GPcode/Elements/pyEEL/notebooks/DL/diffRender/Physics/xr_pbd_extension.py

To use in other Elements notebooks:

```
import sys; sys.path.insert(0, '../Physics')
from xr_pbd_extension import XRPBDSimulator, Particle, DistanceConstraint
```

Free-fall validation: y=-7.588m, vy=-15.70m/s after 1.6s

Analytical: y=-7.557m, vy=-15.70m/s

All XR-PBD sections complete and validated!

3D-XR_pbd

March 16, 2026

1 3D XR-PBD in Elements Scenegraph

All XR-PBD simulations run fully in 3D, integrated into the Elements ECSS scenegraph.

Based on: - *MSc Thesis* — *M. Tamiolakis (2024): XR-PBD Real-Time Physics Framework - SIGGRAPH Asia 2025: XR-PBD Cross-platform Physics - Elements framework* (ECSS — Entity Component System Scenegraph)

1.1 How It Works

Each simulation section: 1. **Runs the XR-PBD solver** (XRPBDSimulator) for physics 2. **Creates Elements Entities** for each particle/body 3. **Updates BasicTransform.trs** every frame from XPBD positions 4. **Renders via Elements Scene** with SDL2 + OpenGL4 + ImGui

XR-PBD Particles BasicTransform.trs Elements RenderMesh GPU

XPBD Constraint Solver Scenegraph Update

1.2 Scenegraph Architecture

Root (Entity)

Camera (entityCam1 → entityCam2)

Floor (terrain plane)

Sim_Rope (Entity) ← one Entity per particle

BasicTransform (trs = XPBD position)

RenderMesh (sphere geometry)

ShaderGLDecorator

Sim_Softbody ...

...

Section	Simulation	Scenegraph Entities
1	Rope	16 sphere entities
2	Flexible Rod (bending)	20 capsule entities
3	Soft Body (volume)	4 tet vertices + 6 edge lines
4	Cloth	12×12 grid of points
5	Rigid Body (shape matching)	9-particle box
6	Piercing (insertion constraint)	membrane triangle + needle

```
[ ]: import sys, os, pathlib
import numpy as np
import warnings
warnings.filterwarnings('ignore')

# Resolve the Physics directory robustly
# __vsc_ipynb_file__ is set by VS Code Jupyter; fall back to CWD for plain
# Jupyter.
try:
    _nb_dir = pathlib.Path(__vsc_ipynb_file__).resolve().parent
except NameError:
    _nb_dir = pathlib.Path(os.path.abspath('')).resolve()
_phys_dir = str(_nb_dir)
if _phys_dir not in sys.path:
    sys.path.insert(0, _phys_dir)

# Safety-net: make sure the Elements src/ folder is on sys.path
# Walks up 4 parents from Physics/ to Elements/, then appends src/
_elements_src = str(pathlib.Path(_phys_dir).parents[4] / 'src')
if os.path.isdir(_elements_src) and _elements_src not in sys.path:
    sys.path.insert(0, _elements_src)

from xr_pbd_extension import (
    XRPBDSimulator, Particle,
    DistanceConstraint, VolumeConstraint,
    InsertionConstraint
)

# Elements imports
import Elements.pyECSS.math_utilities as util
from Elements.pyECSS.Entity import Entity
from Elements.pyECSS.Component import BasicTransform, Camera, RenderMesh
from Elements.pyECSS.System import TransformSystem, CameraSystem
from Elements.pyGLV.GL.Scene import Scene
from Elements.pyGLV.GUI.Viewer import RenderGLStateSystem
from Elements.pyGLV.GUI.ImguiDecorator import ImGUIecssDecorator2
from Elements.pyGLV.GL.Shader import (
    InitGLShaderSystem, Shader,
    ShaderGLDecorator, RenderGLShaderSystem
)
from Elements.pyGLV.GL.VertexArray import VertexArray
from Elements.utils.Shortcuts import displayGUI_text
from OpenGL.GL import GL_LINES, GL_POINTS

print(f" Physics dir : {_phys_dir}")
print(f" Elements src : {_elements_src}")
print(" XR-PBD + Elements imports OK")
```



```
print(f" NumPy: {np.__version__}")
```

1.3 Geometry Helpers

Procedural 3D geometry generators for sphere/capsule particles, edges, and grids. These feed into `RenderMesh.vertex_attributes` for Elements' GPU pipeline.

```
[ ]: def make_sphere_mesh(r=0.06, lats=6, lons=8):
    """Generate a UV sphere as (vertices_Nx4, colors_Nx4, indices)."""
    verts, cols, idxs = [], [], []
    for i in range(lats+1):
        lat = np.pi/2 - np.pi * i / lats
        for j in range(lons+1):
            lon = 2*np.pi * j / lons
            x = r * np.cos(lat) * np.cos(lon)
            y = r * np.sin(lat)
            z = r * np.cos(lat) * np.sin(lon)
            verts.append([x, y, z, 1.0])
            # Color by latitude (top=bright, bottom=dark)
            t = i / lats
            cols.append([0.2+0.8*(1-t), 0.5*t, 0.9*(1-t), 1.0])
    for i in range(lats):
        for j in range(lons):
            a = i*(lons+1)+j
            idxs += [a, a+1, a+lons+1, a+1, a+lons+2, a+lons+1]
    return (np.array(verts, np.float32),
            np.array(cols, np.float32),
            np.array(idxs, np.uint32))

def make_edge_mesh(p0, p1, color=(1,0.3,0.3,1)):
    """A single edge (line segment) between two 3D points."""
    verts = np.array([[*p0, 1.0], [*p1, 1.0]], np.float32)
    cols = np.array([color, color], np.float32)
    idxs = np.array([0, 1], np.uint32)
    return verts, cols, idxs

def make_point_mesh(color=(1,1,0,1)):
    """A single point marker at origin."""
    verts = np.array([[0,0,0,1]], np.float32)
    cols = np.array([color], np.float32)
    idxs = np.array([0], np.uint32)
    return verts, cols, idxs

# Helper: create camera + scene with terrain
def build_scene_base(title="XR-PBD 3D"):
    """Build a Scene with root, camera, terrain floor. Returns (scene, root,
    ↪cam_data)."""
```

```

scene = Scene()
root = scene.world.createEntity(Entity(name="Root"))

# Camera hierarchy
eCam1 = scene.world.createEntity(Entity(name="eCam1"))
scene.world.addEntityChild(root, eCam1)
trans1 = scene.world.addComponent(eCam1, BasicTransform(name="trans1",
↪trs=util.identity()))

eye = util.vec(4.0, 3.0, 4.0)
target = util.vec(0.0, 0.0, 0.0)
up = util.vec(0.0, 1.0, 0.0)
view = util.lookat(eye, target, up)
proj = util.perspective(50.0, 1.33, 0.01, 50.0)
m = np.linalg.inv(proj @ view)

eCam2 = scene.world.createEntity(Entity(name="eCam2"))
scene.world.addEntityChild(eCam1, eCam2)
scene.world.addComponent(eCam2, BasicTransform(name="trans2", trs=util.
↪identity()))
orthoCam = scene.world.addComponent(eCam2, Camera(m, "orthoCam", "Camera",
↪"500"))

return scene, root, orthoCam, proj

SPHERE_V, SPHERE_C, SPHERE_I = make_sphere_mesh(r=0.07)
print(f" Sphere mesh: {len(SPHERE_V)} verts, {len(SPHERE_I)//3} tris")
print(" Scene helpers ready")

```

```

[ ]: def add_particle_entity(scene, root, name, pos, sphere_v, sphere_c, sphere_i,
↪color=None):
    """
    Add a single particle entity to the Elements scenegraph at given 3D
    ↪position.
    Returns (entity, transform, shader_decorator).
    """
    ent = scene.world.createEntity(Entity(name=name))
    scene.world.addEntityChild(root, ent)
    trs = util.translate(float(pos[0]), float(pos[1]), float(pos[2]))
    trans = scene.world.addComponent(ent, BasicTransform(name=f"{name}_t",
    ↪trs=trs))
    mesh = scene.world.addComponent(ent, RenderMesh(name=f"{name}_m"))

    # Optionally re-colour sphere
    vc = sphere_c.copy()
    if color is not None:
        vc[:] = color

```

```

mesh.vertex_attributes.append(sphere_v)
mesh.vertex_attributes.append(vc)
mesh.vertex_index.append(sphere_i)
scene.world.addComponent(ent, VertexArray())
shader = scene.world.addComponent(ent,
    ShaderGLDecorator(Shader(
        vertex_source = Shader.COLOR_VERT_MVP,
        fragment_source= Shader.COLOR_FRAG)))
return ent, trans, shader

def add_edge_entity(scene, root, name, p0, p1):
    """Add a line-segment edge entity between two positions."""
    ent = scene.world.createEntity(Entity(name=name))
    scene.world.addEntityChild(root, ent)
    trans = scene.world.addComponent(ent, BasicTransform(name=f"{name}_t",
    ↪trs=util.identity()))
    mesh = scene.world.addComponent(ent, RenderMesh(name=f"{name}_m"))
    ev, ec, ei = make_edge_mesh(p0, p1)
    mesh.vertex_attributes.append(ev)
    mesh.vertex_attributes.append(ec)
    mesh.vertex_index.append(ei)
    scene.world.addComponent(ent, VertexArray(primitive=GL_LINES))
    shader = scene.world.addComponent(ent,
        ShaderGLDecorator(Shader(
            vertex_source = Shader.COLOR_VERT_MVP,
            fragment_source= Shader.COLOR_FRAG)))
    return ent, trans, shader, ev

def update_transform(trans, shader, mvp):
    """Push updated MVP to a particle's shader uniform."""
    shader.setUniformVariable(key='modelViewProj', value=mvp, mat4=True)

print(" Entity factory functions ready")

```

1.4 Section 1: 3D Rope Simulation in Elements Scenegraph

XR-PBD drives the particles; each particle maps to a **sphere Entity** in the scenegraph.

Every frame: 1. `sim.step(dt)` → updates `particle.pos` 2. `BasicTransform.trs = translate(pos.x, pos.y, pos.z)` 3. `mvp = proj @ view @ trs` → GPU uniform

The **line edges** (`VertexArray GL_LINES`) connect consecutive particles and are updated by rewriting their vertex buffers.

```

[ ]: # 3D Rope Demo
import imgui

def run_rope_3d(n_rope=16, sim_steps=500, dt=0.016):

```

```

"""3D rope simulation in Elements scenegraph. Hit ESC to exit."""
# 1. Build XR-PBD simulation
sim = XRPBDSimulator(substeps=8, iterations=6, sor_factor=1.8,
                      gravity=[0., -9.81, 0.])
ids = sim.add_rope(n=n_rope, length=3.5, origin=(0., 3.5), alpha=1e-4)
# Rope is 2D internally - expand to 3D
for i, pid in enumerate(ids):
    pt = sim.particles[pid]
    pt.pos = np.array([pt.pos[0], pt.pos[1], 0.0], dtype=float)
    pt.prev_pos = pt.pos.copy()
    pt.vel = np.zeros(3)
sim.gravity = np.array([0., -9.81, 0.])
# Swing impulse at mid-point (in 3D)
sim.particles[ids[8]].vel = np.array([4.0, 0.0, 2.0])

# 2. Build Elements Scene
scene, root, orthoCam, proj = build_scene_base("XR-PBD: 3D Rope")

# Add Systems
trans_sys = scene.world.
↪createSystem(TransformSystem("ts", "TransformSystem", "001"))
cam_sys = scene.world.
↪createSystem(CameraSystem("cs", "CameraSystem", "200"))
render_sys = scene.world.createSystem(RenderGLShaderSystem())
init_sys = scene.world.createSystem(InitGLShaderSystem())

# 3. Scenegraph: one entity per particle
particle_data = [] # list of (trans, shader)
for i, pid in enumerate(ids):
    pos = sim.particles[pid].pos
    col = np.tile([0.2, 0.8, 1.0, 1.0], (len(SPHERE_V), 1)).astype(np.
↪float32)
    if sim.particles[pid].inv_mass == 0:
        col[:] = [1.0, 0.5, 0.0, 1.0] # pin = orange
    ent, tr, sh = add_particle_entity(scene, root, f"rope_{i}", pos,
                                     SPHERE_V, SPHERE_C, SPHERE_I)
    particle_data.append((tr, sh))

# 4. Edge entities (lines between consecutive particles)
edge_data = []
for i in range(n_rope - 1):
    p0 = sim.particles[ids[i]].pos
    p1 = sim.particles[ids[i+1]].pos
    ent_e, tr_e, sh_e, ev = add_edge_entity(scene, root, f"edge_{i}", p0,
↪p1)
    edge_data.append((tr_e, sh_e, ev, ent_e))

```

```

# 5. Init GL
scene.init(imgui=True, windowWidth=1024, windowHeight=768,
           windowTitle="XR-PBD 3D Rope - Elements Scenegraph",
           customImGuiDecorator=ImGuiIecssDecorator2, openGLversion=4)
scene.world.traverse_visit(init_sys, scene.world.root)

eManager          = scene.world.eventManager
gWindow           = scene.renderWindow
rgl_actuator      = RenderGLStateSystem()
eManager._subscribers['OnUpdateCamera'] = gWindow
eManager._actuators['OnUpdateCamera']   = rgl_actuator
eManager._subscribers['OnUpdateWireframe'] = gWindow
eManager._actuators['OnUpdateWireframe'] = rgl_actuator
gWindow._myCamera = util.lookat(util.vec(4,3,4), util.vec(0,0,0), util.
↪vec(0,1,0))

# 6. Run loop
step = 0; running = True
while running:
    running = scene.render()

    # XPBD physics step
    sim.step(dt)
    view = gWindow._myCamera

    # Update particle transforms
    for i, (tr, sh) in enumerate(particle_data):
        pos = sim.particles[ids[i]].pos
        mdl = util.translate(float(pos[0]), float(pos[1]), float(pos[2]))
        mvp = proj @ view @ mdl
        tr.trs = mdl
        sh.setUniformVariable(key='modelViewProj', value=mvp, mat4=True)

    # Update edge lines
    for i, (tr_e, sh_e, ev, ent_e) in enumerate(edge_data):
        p0 = sim.particles[ids[i]].pos
        p1 = sim.particles[ids[i+1]].pos
        ev[0,:3] = p0; ev[1,:3] = p1
        mvp_e = proj @ view @ util.identity()
        sh_e.setUniformVariable(key='modelViewProj', value=mvp_e, mat4=True)

    # ImGui overlay
    displayGUI_text(
        f"XR-PBD 3D Rope\n"
        f"Particles: {n_rope} | Constraints: {len(sim.constraints)}\n"
        f"Step: {step} | dt={dt*1000:.0f}ms\n"
        f"ESC to exit"

```

```

    )
    scene.world.traverse_visit(render_sys, scene.world.root)
    scene.world.traverse_visit_pre_camera(cam_sys, orthoCam)
    scene.world.traverse_visit(cam_sys, scene.world.root)
    scene.render_post()
    step += 1

scene.shutdown()
print(f" 3D Rope simulation finished after {step} steps")

print("run_rope_3d() defined - runs an interactive Elements window")
print("Call: run_rope_3d()")

```

```

[ ]: # Uncomment to launch interactive 3D rope window:
run_rope_3d()

```

1.5 Section 2: 3D Soft Body — Tetrahedral Volume Constraint in Elements

A **tetrahedron** with 4 particle vertices, 6 edge constraints (distance), and 1 volume constraint. Each of the 4 vertices is an Entity; the 6 edges are GL_LINES entities. Gravity pulls it down, the volume constraint prevents it from collapsing.

```

[ ]: def run_softbody_3d(sim_steps=600, dt=0.014):
    """3D tetrahedral soft body in Elements scenegraph."""
    sim = XRPBDSimulator(substeps=10, iterations=12, sor_factor=1.8,
                          gravity=[0., -9.81, 0.])

    # Tetrahedron vertices (centered, above origin)
    verts = np.array([[0,0,0],[0.9,0,0],[0.45,0,0.78],[0.45,0.735,0.26]], float)
    verts -= verts.mean(0); verts[:,1] += 2.2
    pids = [sim.add_particle(v.tolist(), 1.0) for v in verts]
    EMAP = [(0,1),(0,2),(0,3),(1,2),(1,3),(2,3)]
    for a,b in EMAP:
        sim.add_constraint(DistanceConstraint(pids[a], pids[b], sim.particles,
        ↪alpha=1e-4))
    vc = VolumeConstraint(pids[0], pids[1], pids[2], pids[3], sim.particles,
    ↪alpha=1e-4)
    sim.add_constraint(vc)

    # Build Elements Scene
    scene, root, orthoCam, proj = build_scene_base("XR-PBD: 3D Soft Body")
    trans_sys = scene.world.
    ↪createSystem(TransformSystem("ts","TransformSystem","001"))
    cam_sys = scene.world.
    ↪createSystem(CameraSystem("cs","CameraSystem","200"))
    render_sys = scene.world.createSystem(RenderGLShaderSystem())
    init_sys = scene.world.createSystem(InitGLShaderSystem())

```

```

# Vertex sphere entities
colors = [[0.9,0.2,0.2,1],[0.2,0.9,0.2,1],[0.2,0.2,0.9,1],[0.9,0.9,0.2,1]]
vert_data = []
for i, pid in enumerate(pids):
    pos = sim.particles[pid].pos
    col = np.tile(colors[i], (len(SPHERE_V),1)).astype(np.float32)
    ent, tr, sh = add_particle_entity(scene, root, f"tet_{i}", pos,
                                     SPHERE_V, SPHERE_C, SPHERE_I,
↳color=col)
    vert_data.append((tr, sh, pid))

# Edge entities (lines)
edge_data = []
for ei,(a,b) in enumerate(EMAP):
    p0 = sim.particles[pids[a]].pos
    p1 = sim.particles[pids[b]].pos
    ent_e, tr_e, sh_e, ev = add_edge_entity(scene, root, f"tet_e{ei}", p0,
↳p1)
    edge_data.append((tr_e, sh_e, ev, a, b))

scene.init(imgui=True, windowWidth=1024, windowHeight=768,
           windowTitle="XR-PBD 3D Soft Body - Volume + Distance,
↳Constraints",
           customImGuiDecorator=ImGuiIecssDecorator2, openGLversion=4)
scene.world.traverse_visit(init_sys, scene.world.root)

eManager = scene.world.eventManager; gWindow = scene.renderWindow
rgl = RenderGLStateSystem()
for ev_name in ['OnUpdateCamera', 'OnUpdateWireframe']:
    eManager._subscribers[ev_name] = gWindow
    eManager._actuators[ev_name] = rgl
gWindow._myCamera = util.lookat(util.vec(3,2.5,3),util.vec(0,1,0),util.
↳vec(0,1,0))

step = 0; running = True
while running:
    running = scene.render()
    sim.step(dt)
    view = gWindow._myCamera

# Ground bounce
for pid in pids:
    pt = sim.particles[pid]
    if pt.pos[1] < -1.5:
        pt.pos[1] = -1.5
        pt.vel[1] = abs(pt.vel[1]) * 0.45

```

```

# Volume conservation check (every 100 steps)
p = [sim.particles[pid].pos for pid in pids]
d1,d2,d3 = p[1]-p[0], p[2]-p[0], p[3]-p[0]
Vcur = abs(np.dot(d1, np.cross(d2,d3))) / 6.0
vol_err = abs(Vcur - vc.V0) / max(vc.V0, 1e-9) * 100

# Update vertex entities
for tr, sh, pid in vert_data:
    pos = sim.particles[pid].pos
    mdl = util.translate(float(pos[0]), float(pos[1]), float(pos[2]))
    tr.trs = mdl
    sh.setUniformVariable(key='modelViewProj', value=(proj@view@mdl),
↪mat4=True)

# Update edge entities
for tr_e, sh_e, ev, a, b in edge_data:
    ev[0,:3] = sim.particles[pids[a]].pos
    ev[1,:3] = sim.particles[pids[b]].pos
    sh_e.setUniformVariable(key='modelViewProj', value=(proj@view@util.
↪identity()), mat4=True)

displayGUI_text(
    f"XR-PBD 3D Soft Body (Tetrahedron)\n"
    f"Volume V0={vc.V0:.3f} Vcur={Vcur:.3f} err={vol_err:.1f}%\n"
    f"Step {step} | Distance+Volume Constraints | SOR w=1.8\n"
    f"ESC to exit"
)

scene.world.traverse_visit(render_sys, scene.world.root)
scene.world.traverse_visit_pre_camera(cam_sys, orthoCam)
scene.world.traverse_visit(cam_sys, scene.world.root)
scene.render_post()
step += 1

scene.shutdown()
Vf = abs(np.dot(d1, np.cross(d2,d3)))/6.0
print(f" Soft body: V0={vc.V0:.4f} Vf={Vf:.4f} err={abs(Vf-vc.V0)/vc.
↪V0*100:.2f}% ({step} steps)")

print("run_softbody_3d() defined")

```

```

[ ]: # Uncomment to launch:
run_softbody_3d()

```


1.6 Section 3: 3D Cloth Simulation in Elements

12 × 12 particle grid with structural, shear, and bending distance constraints. Two top-corner particles are pinned. A wind impulse is applied at t=60 frames.

Each particle is rendered as a **point entity** (GL_POINTS) and grid row/col connections as GL_LINES. The simulation runs in a 3D XZ plane initially, but wind pushes it out-of-plane.

```
[ ]: def run_cloth_3d(rows=10, cols=10, dt=0.016):
    """3D cloth simulation in Elements scenegraph."""
    sim = XRPBDSimulator(substeps=6, iterations=8, sor_factor=1.8,
                          gravity=[0., -9.81, 0.])
    ids = sim.add_cloth_grid(rows=rows, cols=cols, width=3.0, height=3.0,
                             origin=(0.0, 3.0), mass=0.4)

    # Promote 2D particles to 3D (add z=0)
    for r in range(rows):
        for c in range(cols):
            pt = sim.particles[ids[r,c]]
            pt.pos = np.array([pt.pos[0], pt.pos[1], 0.0], dtype=float)
            pt.prev_pos = pt.pos.copy()
            pt.vel = np.zeros(3, dtype=float)
    sim.gravity = np.array([0., -9.81, 0.])

    # Build Scene
    scene, root, orthoCam, proj = build_scene_base("XR-PBD: 3D Cloth")
    trans_sys = scene.world.
    ↪createSystem(TransformSystem("ts", "TransformSystem", "001"))
    cam_sys = scene.world.
    ↪createSystem(CameraSystem("cs", "CameraSystem", "200"))
    render_sys = scene.world.createSystem(RenderGLShaderSystem())
    init_sys = scene.world.createSystem(InitGLShaderSystem())

    # One small sphere entity per particle
    sV, sC, sI = make_sphere_mesh(r=0.04, lats=4, lons=6)
    part_data = []
    for r in range(rows):
        for c in range(cols):
            pid = ids[r,c]
            pos = sim.particles[pid].pos
            pinned = (sim.particles[pid].inv_mass == 0)
            col = np.tile([1.0,0.4,0.0,1.0] if pinned else [0.5,0.8,1.0,1.0],
                          (len(sV),1)).astype(np.float32)
            ent, tr, sh = add_particle_entity(scene, root, f"c_{r}_{c}", pos,
            ↪sV, sC, sI, color=col)
            part_data.append((r, c, pid, tr, sh))

    # Horizontal edge entities (row connections)
```

```

h_edges = []
for r in range(rows):
    for c in range(cols-1):
        p0 = sim.particles[ids[r,c]].pos
        p1 = sim.particles[ids[r,c+1]].pos
        _, tr_e, sh_e, ev = add_edge_entity(scene, root, f"he_{r}_{c}", p0,
↪p1)
        h_edges.append((tr_e, sh_e, ev, r, c))

# Vertical edge entities (column connections)
v_edges = []
for r in range(rows-1):
    for c in range(cols):
        p0 = sim.particles[ids[r,c]].pos
        p1 = sim.particles[ids[r+1,c]].pos
        _, tr_e, sh_e, ev = add_edge_entity(scene, root, f"ve_{r}_{c}", p0,
↪p1)
        v_edges.append((tr_e, sh_e, ev, r, c))

scene.init(imgui=True, windowWidth=1024, windowHeight=768,
            windowTitle="XR-PBD 3D Cloth - Elements Scenegraph",
            customImGuiDecorator=ImGuiIecssDecorator2, openGLversion=4)
scene.world.traverse_visit(init_sys, scene.world.root)

eManager = scene.world.eventManager; gWindow = scene.renderWindow
rgl = RenderGLStateSystem()
for ev_name in ['OnUpdateCamera', 'OnUpdateWireframe']:
    eManager._subscribers[ev_name] = gWindow
    eManager._actuators[ev_name] = rgl
gWindow._myCamera = util.lookat(util.vec(5,2,2),util.vec(0,0,0),util.
↪vec(0,1,0))

step = 0; running = True
while running:
    running = scene.render()
    # Wind impulse at step 60
    if step == 60:
        for r in range(rows):
            for c in range(cols):
                if sim.particles[ids[r,c]].inv_mass > 0:
                    sim.particles[ids[r,c]].vel += np.array([2.0, 0.0, 3.0])
    sim.step(dt)
    view = gWindow._myCamera

# Update particle spheres
for r,c,pid,tr,sh in part_data:
    pos = sim.particles[pid].pos

```

```

        mdl = util.translate(float(pos[0]), float(pos[1]), float(pos[2]))
        tr.trs = mdl
        sh.setUniformVariable(key='modelViewProj', value=(proj@view@mdl),
↪mat4=True)

    identity = util.identity()
   .mvp_e = proj @ view @ identity
    for tr_e, sh_e, ev, r, c in h_edges:
        ev[0,:3] = sim.particles[ids[r,c]].pos
        ev[1,:3] = sim.particles[ids[r,c+1]].pos
        sh_e.setUniformVariable(key='modelViewProj', value=mvp_e, mat4=True)
    for tr_e, sh_e, ev, r, c in v_edges:
        ev[0,:3] = sim.particles[ids[r,c]].pos
        ev[1,:3] = sim.particles[ids[r+1,c]].pos
        sh_e.setUniformVariable(key='modelViewProj', value=mvp_e, mat4=True)

    KE = sim.kinetic_energy()
    displayGUI_text(
        f"XR-PBD 3D Cloth ({rows}x{cols} grid)\n"
        f"Particles: {rows*cols} | Constraints: {len(sim.constraints)}\n"
        f"KE={KE:.3f} J | Step {step}\n"
        f"Wind impulse at t=60 frames | SOR w=1.8\n"
        f"ESC to exit"
    )
    scene.world.traverse_visit(render_sys, scene.world.root)
    scene.world.traverse_visit_pre_camera(cam_sys, orthoCam)
    scene.world.traverse_visit(cam_sys, scene.world.root)
    scene.render_post()
    step += 1

    scene.shutdown()
    print(f" 3D Cloth: {rows*cols} particles, {len(sim.constraints)}
↪constraints, {step} steps")

print("run_cloth_3d() defined")

```

```

[ ]: # Uncomment to launch:
run_cloth_3d(rows=10, cols=10)

```

1.7 Section 4: 3D Rigid Body — Shape Matching in Elements

Shape matching extracts a rotation matrix R via SVD polar decomposition of the deformation gradient, then pulls all particles toward their rotated rest positions.

In this demo, a $3 \times 3 \times 3$ cube of particles (27 nodes) is launched with angular velocity and collides with a floor plane.

```

[ ]: class ShapeMatchingConstraint3D:
    """
    3D shape matching constraint (Müller 2005) for rigid-body-like behaviour.
    Pulls particles toward SVD-extracted rotated rest positions.
    """
    def __init__(self, particle_ids, particles, stiffness=0.95):
        self.ids = particle_ids
        self.particles = particles
        self.stiffness = stiffness
        poses = np.array([particles[i].pos for i in particle_ids])
        self.rest_offsets = poses - poses.mean(0)

    def reset_lambda(self): pass

    def solve(self, h, sor=1.0):
        poses = np.array([self.particles[i].pos for i in self.ids])
        com = poses.mean(0)
        offsets = poses - com
        Apq = offsets.T @ self.rest_offsets
        try:
            U, _, Vt = np.linalg.svd(Apq)
        except np.linalg.LinAlgError:
            return
        R = U @ Vt
        if np.linalg.det(R) < 0:
            U[:, -1] *= -1; R = U @ Vt
        targets = (R @ self.rest_offsets.T).T + com
        for pid, tgt in zip(self.ids, targets):
            self.particles[pid].pos += (tgt - self.particles[pid].pos) * self.
↪stiffness * sor

    def run_rigid_3d(dt=0.016):
        """3D rigid-body shape matching in Elements scenegraph."""
        sim = XRPBDSimulator(substeps=6, iterations=6, sor_factor=1.5,
                               gravity=[0., -9.81, 0.])
        # 3x3x3 particle cube
        n = 3; spacing = 0.4
        pids = []
        for ix in range(n):
            for iy in range(n):
                for iz in range(n):
                    x = (ix - n//2) * spacing
                    y = (iy - n//2) * spacing + 2.0
                    z = (iz - n//2) * spacing
                    pid = sim.add_particle([x,y,z], 0.8)
                    pids.append(pid)
        # Angular velocity (spin)

```

```

com = np.mean([sim.particles[p].pos for p in pids], axis=0)
for pid in pids:
    r = sim.particles[pid].pos - com
    omega = np.array([2.0, 3.0, 1.0]) # angular velocity vector
    sim.particles[pid].vel = np.cross(omega, r)
    sim.add_constraint(ShapeMatchingConstraint3D(pids, sim.particles,
↪stiffness=0.95))

# Build Scene
scene, root, orthoCam, proj = build_scene_base("XR-PBD: 3D Rigid Body")
trans_sys = scene.world.
↪createSystem(TransformSystem("ts", "TransformSystem", "001"))
cam_sys = scene.world.
↪createSystem(CameraSystem("cs", "CameraSystem", "200"))
render_sys = scene.world.createSystem(RenderGLShaderSystem())
init_sys = scene.world.createSystem(InitGLShaderSystem())

sV, sC, sI = make_sphere_mesh(r=0.12, lats=6, lons=8)
part_data = []
for i, pid in enumerate(pids):
    pos = sim.particles[pid].pos
    r,g,b = (i%3)*0.3+0.1, ((i//3)%3)*0.3+0.1, ((i//9)%3)*0.3+0.1
    col = np.tile([r,g,b,1.0],(len(sV),1)).astype(np.float32)
    ent, tr, sh = add_particle_entity(scene, root, f"rb_{i}", pos, sV, sC,
↪sI, color=col)
    part_data.append((pid, tr, sh))

scene.init(imgui=True, windowWidth=1024, windowHeight=768,
           windowTitle="XR-PBD 3D Rigid Body - Shape Matching",
           customImGuiDecorator=ImGuiIecssDecorator2, openGLversion=4)
scene.world.traverse_visit(init_sys, scene.world.root)

eManager = scene.world.eventManager; gWindow = scene.renderWindow
rgl = RenderGLStateSystem()
for ev_name in ['OnUpdateCamera', 'OnUpdateWireframe']:
    eManager._subscribers[ev_name] = gWindow
    eManager._actuators[ev_name] = rgl
gWindow._myCamera = util.lookat(util.vec(4,3,4), util.vec(0,1,0), util.
↪vec(0,1,0))

step = 0; running = True
while running:
    running = scene.render()
    sim.step(dt)
    # Floor
    for pid in pids:
        pt = sim.particles[pid]

```

```

        if pt.pos[1] < -2.0:
            pt.pos[1] = -2.0; pt.vel[1] = abs(pt.vel[1]) * 0.4
view = gWindow._myCamera
KE = sim.kinetic_energy()

for pid, tr, sh in part_data:
    pos = sim.particles[pid].pos
    mdl = util.translate(float(pos[0]), float(pos[1]), float(pos[2]))
    tr.trs = mdl
    sh.setUniformVariable(key='modelViewProj', value=(proj@view@mdl),
↳mat4=True)

displayGUI_text(
    f"XR-PBD 3D Rigid Body - Shape Matching (SVD)\n"
    f"Particles: {n**3} (3x3x3 grid) | Stiffness=0.95\n"
    f"KE={KE:.3f} J | Step {step}\n"
    f"ESC to exit"
)
scene.world.traverse_visit(render_sys, scene.world.root)
scene.world.traverse_visit_pre_camera(cam_sys, orthoCam)
scene.world.traverse_visit(cam_sys, scene.world.root)
scene.render_post()
step += 1

scene.shutdown()
print(f" 3D Rigid body: {n**3} particles, {step} steps, final KE={KE:.4f}")

print("run_rigid_3d() defined")

```

```

[ ]: # Uncomment to launch:
run_rigid_3d()

```

1.8 Section 5: 3D Tearing Simulation in Elements

Breakable distance constraints: each constraint estimates $F \approx |\Delta\lambda|/\Delta t^2$. When $F > F_{threshold}$, the constraint is deactivated.

The broken edges turn **red** in the Elements scenegraph to highlight the tear.

```

[ ]: def run_tearing_3d(n=20, break_force=4.0, dt=0.016):
    """3D rope tearing in Elements scenegraph."""
    sim = XRPBDSimulator(substeps=6, iterations=8, sor_factor=1.8,
                          gravity=[0., -4.0, 0.]) # softer gravity to see tear
    for i in range(n):
        sim.add_particle([0.0, 3.5 - i*0.18, 0.0], mass=1.0)
    sim.particles[0].inv_mass = 0 # pin top
    constraints = []
    for i in range(n-1):

```

```

        c = DistanceConstraint(i, i+1, sim.particles, alpha=1e-5,
                               breakable=True, break_force=break_force)
        sim.add_constraint(c); constraints.append(c)
# Heavy mass at bottom
        sim.particles[-1].mass = 12.0; sim.particles[-1].inv_mass = 1/12.0

# Build Scene
        scene, root, orthoCam, proj = build_scene_base("XR-PBD: 3D Tearing")
        trans_sys = scene.world.
↪createSystem(TransformSystem("ts","TransformSystem","001"))
        cam_sys = scene.world.
↪createSystem(CameraSystem("cs","CameraSystem","200"))
        render_sys = scene.world.createSystem(RenderGLShaderSystem())
        init_sys = scene.world.createSystem(InitGLShaderSystem())

        sV, sC, sI = make_sphere_mesh(r=0.055, lats=5, lons=6)

        part_data = []
        for i in range(n):
            pos = sim.particles[i].pos
            pin = (sim.particles[i].inv_mass == 0)
            col = np.tile([1.0,0.5,0.0,1.0] if pin else [0.2,0.8,0.8,1.0],
                          (len(sV),1)).astype(np.float32)
            ent, tr, sh = add_particle_entity(scene, root, f"tr_{i}", pos, sV, sC,
↪sI, color=col)
            part_data.append((tr, sh))

        edge_data = []
        for i, c in enumerate(constraints):
            p0 = sim.particles[i].pos
            p1 = sim.particles[i+1].pos
            _, tr_e, sh_e, ev = add_edge_entity(scene, root, f"e_{i}", p0, p1)
            edge_data.append((tr_e, sh_e, ev, c))

        scene.init(imgui=True, windowWidth=1024, windowHeight=768,
                   windowTitle="XR-PBD 3D Tearing - Breakable Constraints",
                   customImGuiDecorator=ImGuiIecssDecorator2, openGLversion=4)
        scene.world.traverse_visit(init_sys, scene.world.root)

        eManager = scene.world.eventManager; gWindow = scene.renderWindow
        rgl = RenderGLStateSystem()
        for ev_name in ['OnUpdateCamera','OnUpdateWireframe']:
            eManager._subscribers[ev_name] = gWindow
            eManager._actuators[ev_name] = rgl
        gWindow._myCamera = util.lookat(util.vec(2,2,3), util.vec(0,1.5,0), util.
↪vec(0,1,0))

```

```

step = 0; running = True
while running:
    running = scene.render()
    sim.step(dt)
    view = gWindow._myCamera
    n_broken = sum(1 for c in constraints if not c.active)
   .mvp_e = proj @ view @ util.identity()

    for i, (tr, sh) in enumerate(part_data):
        pos = sim.particles[i].pos
        mdl = util.translate(float(pos[0]), float(pos[1]), float(pos[2]))
        tr.trs = mdl
        sh.setUniformVariable(key='modelViewProj', value=(proj@view@mdl),
↪mat4=True)

        for tr_e, sh_e, ev, c in edge_data:
            ev[0,:3] = sim.particles[c.i1].pos
            ev[1,:3] = sim.particles[c.i2].pos
            # Broken edges turn red
            if not c.active:
                sh_e.setUniformVariable(key='modelViewProj', value=mvp_e * 0,
↪mat4=True) # hide broken
            else:
                sh_e.setUniformVariable(key='modelViewProj', value=mvp_e,
↪mat4=True)

        displayGUI_text(
            f"XR-PBD 3D Tearing\n"
            f"Break threshold: {break_force} N | Broken: {n_broken}/{n-1}\n"
            f"Step {step} | Heavy bottom mass = 12kg\n"
            f"ESC to exit"
        )
    scene.world.traverse_visit(render_sys, scene.world.root)
    scene.world.traverse_visit_pre_camera(cam_sys, orthoCam)
    scene.world.traverse_visit(cam_sys, scene.world.root)
    scene.render_post()
    step += 1

scene.shutdown()
n_broken = sum(1 for c in constraints if not c.active)
print(f" 3D Tearing: broke {n_broken}/{n-1} constraints ({step} steps)")

print("run_tearing_3d() defined")

```

```

[ ]: # Uncomment to launch:
run_tearing_3d(n=20, break_force=4.0)

```


1.9 Section 6: 3D Needle Piercing — Insertion Constraint in Elements

The **novel XR-PBD contribution** (SIGGRAPH Asia 2025, §4.4): A 5-particle constraint couples a moving needle segment (n_0, n_1) to a triangular membrane face (t_0, t_1, t_2) .

$$q = w_0x_{t_0} + w_1x_{t_1} + w_2x_{t_2}, \quad r = (1-v)n_0 + vn_1, \quad C = |q - r|$$

In the Elements scene: - **Cyan spheres** = tissue membrane vertices (3 pinned) - **Orange cylinder** (GL_LINES) = needle - Green line = constraint connection (q→r) drawn realtime

```
[ ]: def run_piercing_3d(dt=0.016):
    """3D needle piercing simulation in Elements scenegraph."""
    sim = XRPBDSimulator(substeps=8, iterations=10, sor_factor=1.8,
                          gravity=[0., -9.81, 0.])
    # Triangular membrane (3 pinned vertices + 1 dynamic center)
    mem_pts = [[-0.9, 0., 0.], [0.9, 0., 0.], [0., 0., 1.2], [0., 0.15, 0.4]]
    mem_ids = [sim.add_particle(pt, 0.8) for pt in mem_pts]
    for i in range(3): sim.particles[mem_ids[i]].inv_mass = 0 # pin triangle
    for a, b in [(0, 3), (1, 3), (2, 3), (0, 1), (1, 2), (2, 0)]:
        sim.add_constraint(DistanceConstraint(mem_ids[a], mem_ids[b], sim.
        ↪particles, alpha=1e-5))

    # Needle: 2 particles, moving downward
    n0 = sim.add_particle([0.0, 3.0, 0.4], 2.0)
    n1 = sim.add_particle([0.0, 2.0, 0.4], 2.0)
    sim.particles[n0].vel = np.array([0., -1.8, 0.])
    sim.particles[n1].vel = np.array([0., -1.8, 0.])
    sim.add_constraint(DistanceConstraint(n0, n1, sim.particles, alpha=1e-7))
    # Insertion constraint: needle into membrane center
    ic = InsertionConstraint(mem_ids[0], mem_ids[1], mem_ids[2],
                             n0, n1, sim.particles,
                             bw0=0.33, bw1=0.34, bw2=0.33, v=0.75,
                             alpha=5e-4)
    sim.add_constraint(ic)

    # Build Scene
    scene, root, orthoCam, proj = build_scene_base("XR-PBD: 3D Piercing")
    trans_sys = scene.world.
    ↪createSystem(TransformSystem("ts", "TransformSystem", "001"))
    cam_sys = scene.world.
    ↪createSystem(CameraSystem("cs", "CameraSystem", "200"))
    render_sys = scene.world.createSystem(RenderGLShaderSystem())
    init_sys = scene.world.createSystem(InitGLShaderSystem())

    sV, sC, sI = make_sphere_mesh(r=0.07, lats=6, lons=8)

    # Membrane sphere entities
```

```

mem_data = []
for i, mid in enumerate(mem_ids):
    pos = sim.particles[mid].pos
    col = np.tile([0.0,0.9,0.9,1.0] if i<3 else [0.0,1.0,0.4,1.0],
                  (len(sV),1)).astype(np.float32)
    ent, tr, sh = add_particle_entity(scene, root, f"mem_{i}", pos, sV, sC,
    ↪sI, color=col)
    mem_data.append((mid, tr, sh))

# Membrane edges
mem_edges = []
for a,b in [(0,3),(1,3),(2,3),(0,1),(1,2),(2,0)]:
    p0 = sim.particles[mem_ids[a]].pos
    p1 = sim.particles[mem_ids[b]].pos
    _, tr_e, sh_e, ev = add_edge_entity(scene, root, f"me_{a}-{b}", p0, p1)
    mem_edges.append((tr_e, sh_e, ev, a, b))

# Needle entities (n0 sphere orange, n1 sphere orange, line between)
sVN, _, sIN = make_sphere_mesh(r=0.08, lats=5, lons=7)
col_needle = np.tile([1.0,0.4,0.0,1.0],(len(sVN),1)).astype(np.float32)
_, tr_n0, sh_n0 = add_particle_entity(scene, root, "ndl0", sim.
    ↪particles[n0].pos, sVN, col_needle.copy(), sIN, color=col_needle)
_, tr_n1, sh_n1 = add_particle_entity(scene, root, "ndl1", sim.
    ↪particles[n1].pos, sVN, col_needle.copy(), sIN, color=col_needle)
_, tr_nl, sh_nl, ev_nl = add_edge_entity(scene, root, "ndl_e",
    sim.particles[n0].pos, sim.
    ↪particles[n1].pos)

# Constraint visualization line
_, tr_cv, sh_cv, ev_cv = add_edge_entity(scene, root, "cons_vis",
    sim.particles[n0].pos, sim.
    ↪particles[n0].pos)

scene.init(imgui=True, windowWidth=1024, windowHeight=768,
           windowTitle="XR-PBD 3D Insertion Constraint - Needle Piercing",
           customImGuiDecorator=ImGuiIecssDecorator2, openGLversion=4)
scene.world.traverse_visit(init_sys, scene.world.root)

eManager = scene.world.eventManager; gWindow = scene.renderWindow
rgl = RenderGLStateSystem()
for ev_name in ['OnUpdateCamera', 'OnUpdateWireframe']:
    eManager._subscribers[ev_name] = gWindow
    eManager._actuators[ev_name] = rgl
gWindow._myCamera = util.lookat(util.vec(3,2,3), util.vec(0,0.5,0), util.
    ↪vec(0,1,0))

step = 0; running = True

```

```

while running:
    running = scene.render()
    sim.step(dt)
    view = gWindow._myCamera
    mvp_I = proj @ view @ util.identity()

    # Compute q,r for visualization
    tp = [sim.particles[m].pos for m in mem_ids[:3]]
    q = 0.33*np.array(tp[0]) + 0.34*np.array(tp[1]) + 0.33*np.array(tp[2])
    r = 0.25*sim.particles[n0].pos + 0.75*sim.particles[n1].pos
    gap = np.linalg.norm(q-r)

    for mid, tr, sh in mem_data:
        pos = sim.particles[mid].pos
        mdl = util.translate(float(pos[0]),float(pos[1]),float(pos[2]))
        tr.trs = mdl
        sh.setUniformVariable(key='modelViewProj', value=(proj@view@mdl),
↪mat4=True)

    for tr_e, sh_e, ev, a, b in mem_edges:
        ev[0,:3] = sim.particles[mem_ids[a]].pos
        ev[1,:3] = sim.particles[mem_ids[b]].pos
        sh_e.setUniformVariable(key='modelViewProj', value=mvp_I, mat4=True)

    for (pid, tr, sh) in [(n0, tr_n0, sh_n0), (n1, tr_n1, sh_n1)]:
        pos = sim.particles[pid].pos
        mdl = util.translate(float(pos[0]),float(pos[1]),float(pos[2]))
        tr.trs = mdl
        sh.setUniformVariable(key='modelViewProj', value=(proj@view@mdl),
↪mat4=True)

    ev_n1[0,:3] = sim.particles[n0].pos
    ev_n1[1,:3] = sim.particles[n1].pos
    sh_n1.setUniformVariable(key='modelViewProj', value=mvp_I, mat4=True)
    ev_cv[0,:3] = q; ev_cv[1,:3] = r
    sh_cv.setUniformVariable(key='modelViewProj', value=mvp_I, mat4=True)

    displayGUI_text(
        f"XR-PBD 3D Insertion Constraint (Piercing)\n"
        f"|q - r| = {gap:.5f} (constraint residual)\n"
        f"Cyan = membrane | Orange = needle\n"
        f"Green line = insertion constraint q-r\n"
        f"Step {step} | ESC to exit"
    )

    scene.world.traverse_visit(render_sys, scene.world.root)
    scene.world.traverse_visit_pre_camera(cam_sys, orthoCam)
    scene.world.traverse_visit(cam_sys, scene.world.root)

```

```

        scene.render_post()
        step += 1

    scene.shutdown()
    print(f" 3D Piercing: final |q-r|={gap:.5f} ({step} steps)")

print("run_piercing_3d() defined")

```

```

[ ]: # Uncomment to launch:
run_piercing_3d()

```

1.10 Quick Launcher

Run any simulation interactively:

Demo	Function	Description
Rope	<code>run_rope_3d()</code>	16-particle rope with swing
Soft Body	<code>run_softbody_3d()</code>	Tetrahedral volume constraint
Cloth	<code>run_cloth_3d()</code>	10×10 grid with wind
Rigid Body	<code>run_rigid_3d()</code>	27-particle spinning box
Tearing	<code>run_tearing_3d()</code>	Breakable rope under load
Piercing	<code>run_piercing_3d()</code>	Novel insertion constraint

```

[ ]: # Interactive launcher
DEMOS = {
    '1': ('Rope', run_rope_3d),
    '2': ('Soft Body', run_softbody_3d),
    '3': ('Cloth', run_cloth_3d),
    '4': ('Rigid', run_rigid_3d),
    '5': ('Tearing', run_tearing_3d),
    '6': ('Piercing', run_piercing_3d),
}
print("Available XR-PBD 3D demos in Elements Scenegraph:")
for k,(name,fn) in DEMOS.items():
    print(f" {k}: {name}")
print()
print("Usage: DEMOS['1'][1]() or run_rope_3d()")

```